Nome: Eduardo de Jesus Torres Souza

Teste 1: Exibição

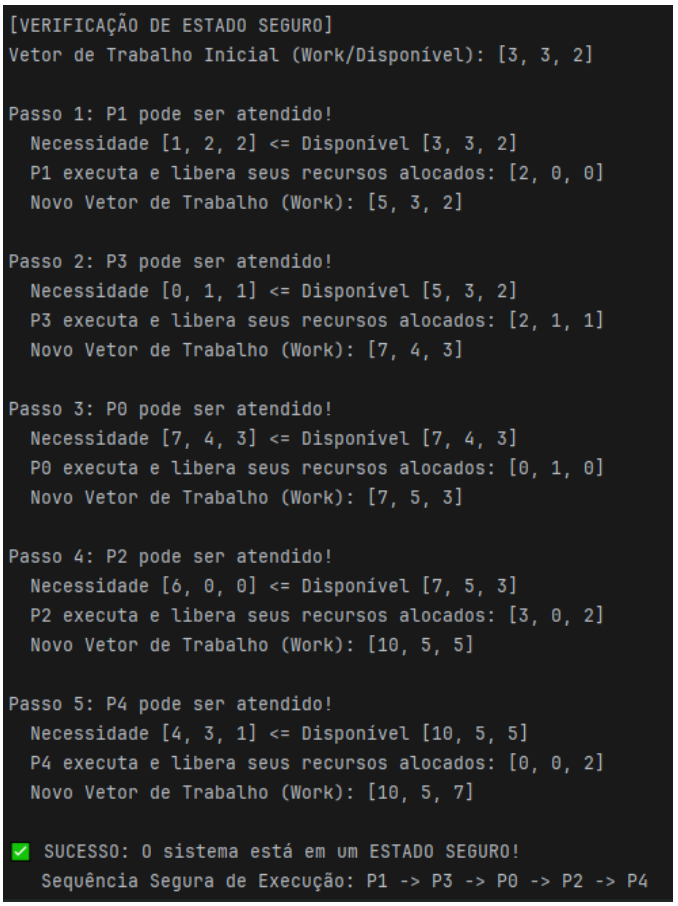
```
Nome Aluno: Eduardo de Jesus Torres Souza
================================================================================
     SIMULAÇÃO DO ALGORITMO DO BANQUEIRO (TANENBAUM & BOS - CAP 6)
================================================================================
================================================================================
ESTADO ATUAL DO SISTEMA
================================================================================
Recursos Totais (E):     [10, 5, 7]
Recursos Disponíveis (A): [3, 3, 2]
--------------------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
--------------------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [2, 0, 0]       | [3, 2, 2]       | [1, 2, 2]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
================================================================================

[VERIFICAÇÃO DE ESTADO SEGURO]
Vetor de Trabalho Inicial (Work/Disponível): [3, 3, 2]
```

O primeiro teste exibe as matrizes que representam os recursos e as requisições dos recursos de cada processo.

- **Alocado:** representa a quantidade de tipos de recursos que já estão sobre a posse de algum processo. (P0 possui uma unidade do processo R1 [0, 1, 0])
- **Necessidade Máxima:** Representa a quantidade máxima de recursos que um processo pode alocar. (P0 vai precisar do total de recursos: [7, 5, 3])
- **Necessidade:** Representa a quantidade de recursos que cada processo ainda falta para terminar a execução. (P0 já tem [0, 1, 0], faltando apenas [7, 4, 3])

Teste 2: Verificação



O segundo teste verifica se o estado inicial possui recursos suficientes para atender as necessidades de cada processo. Além disso, busca uma sequência de execução para que todos os processos terminem sem travamentos.

- **Passo 1:** Escolhe o processo P1 para iniciar a sequência pois necessita de uma quantidade disponível de recursos (Necessita: [1, 2, 2] <= Disponível [3, 3, 2]).
  Após isso, o processo libera os recursos usados junto com os recursos já alocados e disponibiliza para o próximo processo. (Disponível: [5, 3, 2])
- **Passo 2:** O próximo processo P3 passa pela mesma verificação com a nova quantidade de recursos. (Necessita: [0, 1, 1] <= Disponível: [5, 3, 2]).
  Após o término, os recursos alocados em P3 ficam disponíveis para o próximo processo. (Disponível: [7, 4, 3]).
- A verificação se repete da mesma forma até o fim, chegando a um estado seguro com a seguinte sequência: P1 -> P3 -> P0 -> P2 -> P4

Teste 3: caso 1

```
######################################################################
SOLICITAÇÃO DE RECURSOS PELO PROCESSO P1: [1, 0, 2]
######################################################################
⚠️ TENTATIVA PROVISÓRIA: Recursos disponíveis no momento. Testando alocação...

[VERIFICAÇÃO DE ESTADO SEGURO]
Vetor de Trabalho Inicial (Work/Disponível): [2, 3, 0]

Passo 1: P1 pode ser atendido!
  Necessidade [0, 2, 0] <= Disponível [2, 3, 0]
  P1 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [5, 3, 2]

Passo 2: P3 pode ser atendido!
  Necessidade [0, 1, 1] <= Disponível [5, 3, 2]
  P3 executa e libera seus recursos alocados: [2, 1, 1]
  Novo Vetor de Trabalho (Work): [7, 4, 3]

Passo 3: P0 pode ser atendido!
  Necessidade [7, 4, 3] <= Disponível [7, 4, 3]
  P0 executa e libera seus recursos alocados: [0, 1, 0]
  Novo Vetor de Trabalho (Work): [7, 5, 3]

Passo 4: P2 pode ser atendido!
  Necessidade [6, 0, 0] <= Disponível [7, 5, 3]
  P2 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 5]
```

```
Passo 5: P4 pode ser atendido!
  Necessidade [4, 3, 1] <= Disponível [10, 5, 5]
  P4 executa e libera seus recursos alocados: [0, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 7]

✅ SUCESSO: O sistema está em um ESTADO SEGURO!
   Sequência Segura de Execução: P1 -> P3 -> P0 -> P2 -> P4

✅ REQUISIÇÃO CONCEDIDA A P1!
   O estado resultante permanece SEGURO.
======================================================================
ESTADO ATUAL DO SISTEMA
======================================================================
Recursos Totais (E):     [10, 5, 7]
Recursos Disponíveis (A): [2, 3, 0]
----------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
----------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [3, 0, 2]       | [3, 2, 2]       | [0, 2, 0]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
======================================================================
```

O processo P1 solicita recursos a mais, sendo a quantidade de recursos: [1, 0, 2]
O teste do caso 1 é verificar se esse solicitação pode ser aceita ou não com base na seguinte avaliação:

- Verifica se a requisição é menor ou igual a necessidade de recursos que P1 precisava ([1, 0, 2] < [1, 2, 2])
- Sabendo que a requisição é menor que a necessidade de P1, verifica se a requisição é menor que os recursos disponíveis ([1, 0, 2] < [3, 3, 2])
- O sistema simula a alocação dos recursos requisitados de P1, ficando [3, 0, 2] e a necessidade cai para [0, 2, 0], além de subtrair os recursos disponíveis, ficando [2, 3, 0].
- A simulação é rodada para garantir que o sistema não vai travar em algum processo. No caso, ele descobre que mesmo com os recursos disponíveis reduzidos o sistema funciona normalmente sem travamentos (deadlock).

Teste 4: Caso 2

```
######################################################################
SOLICITAÇÃO DE RECURSOS PELO PROCESSO P4: [3, 3, 0]
######################################################################
⏳ PROCESSO BLOQUEADO: Recursos insuficientes disponíveis no momento.
   Solicitado: 3 > Disponível 2 para o recurso R0
   P4 deve aguardar a liberação de recursos por outros processos.
======================================================================
ESTADO ATUAL DO SISTEMA
======================================================================
Recursos Totais (E):     [10, 5, 7]
Recursos Disponíveis (A): [2, 3, 0]
----------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
----------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [3, 0, 2]       | [3, 2, 2]       | [0, 2, 0]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
======================================================================
```

O processo P4 solicita recursos a mais, sendo essa quantidade de recursos: [3, 3, 0]
Assim como no caso 1, o caso 2 verifica se a solicitação pode ser aceita ou não, porém ao contrário do caso 1, o caso 2 não foi aceito e o processo foi bloqueado. A solicitação exigia 3 recursos R0, porém estavam disponíveis apenas 2 dos recursos R0.
(O sistema utilizou os recursos disponíveis da última solicitação no caso 1, por isso os recursos são [2, 3, 0])